MSA 8770 Enterprise Analytics: Project Milestone 2
Instructor: Dr. Saber Soleymani
Group 4 Team: Mi Le, Tran Le, Maria Malhotra, Danny McIntosh, Maya Patel

Project Name: AI-Assisted College Admissions Process

I. Updated Scope & Changes

The project focuses on creating a system that could improve the college admissions application review process at a university. The system assists admissions officers by ingesting large amounts of application materials, extracting and organizing applicant information, checking applications against requirements and policies, and generating a structured, evidence-based review analysis of each application. The primary user is the admissions officer, who reviews the AI-assisted dossier, verifies the supporting evidence, evaluates materials reserved for human review (i.e., personal essay), and determines the appropriate next step toward a final admissions decision. The intended outcome is to reduce the time and manual workload required to prepare applications for review while enhancing consistency and traceability across the full process. The team will evaluate this using measures such as routing correctness, extraction and validation accuracy, review and citation accuracy, processing time vs human, and exception rates.

Since Milestone 1, the project has transitioned from an architecture plan to a working end-to-end prototype. The team has implemented a two-pass deterministic intake process that stages applicant records, associates supporting documents, validates application completeness, and routes eligible applications to the AI processing pipeline. The AI component utilizes a locally hosted Qwen3-VL vision-language model together with retrieval-augmented generation (RAG) over institutional admissions policies to produce structured, evidence-grounded applicant dossiers. Personal essays and personal identifiable information remain excluded from AI evaluation, and generated outputs are validated before being made available for review.

The team also implemented an Applicant Review Chat UI that provides admissions officers with access to applicant information, generated dossiers, supporting evidence, original documents, and an applicant-specific question-and-answer assistant. This interface serves as the human-in-the-loop review layer while preserving the admissions officer's responsibility for judgment and final decision-making.

All current project artifacts and code are developed and maintained in the team's GitHub repository (https://github.com/kappasprouts/MSA8770-Fall2026), with the current implementation merged into the main branch. An invite has been sent to Dr. Soleymani's profile (noghte) from owner kappa_sprouts.0m@icloud.com (Maria's alias/personal email).

II. Updated Workflow

The system workflow prepares applications for human officer's review. Programmed rules (codes) control processing while AI supports extraction and summarization and the human officer retains final admission authority.

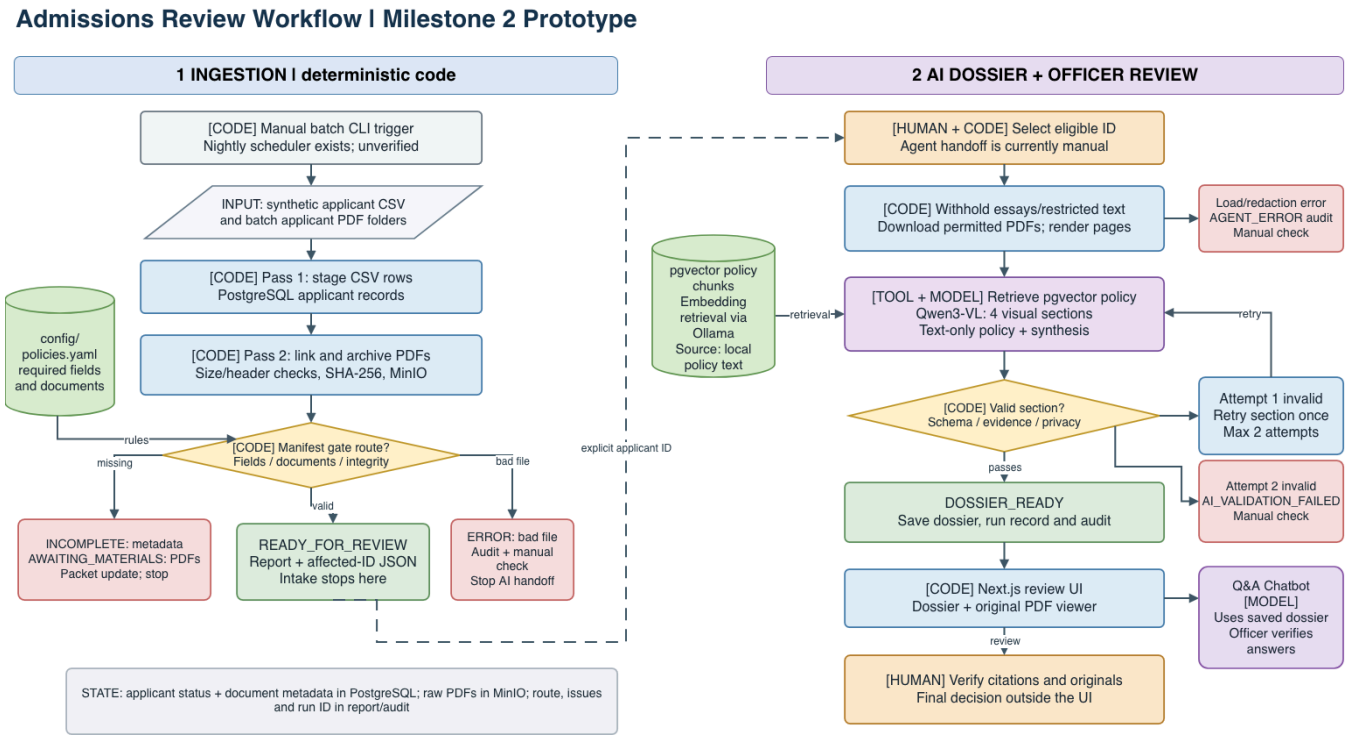
The trigger is a timed automated nightly trigger using APScheduler; although manual triggered runs are available for testing purposes. Ordinary Python codes control a fixed sequence and conditional branches. No model selects the next tool. This fits intake because document presence, filenames, file sizes, and required metadata can be checked consistently without model judgment.

The intake and ingestion connects applicant records with supporting documents. The storage layer is designed to keep structured records and document metadata in PostgreSQL and the original document files in MinIO. Deterministic checks assess required information, document presence, and basic file integrity. "Ready for Review" packets become eligible for review and analysis by the AI agent. Missing documents packets are marked for missing-material follow-up. Invalid packets (failures or errors) are

marked for human review. These routes identify the next action. The notifications and email generation remain planned.

The AI summary agent renders PDFs as images, uses a vision language model (Qwen3-VL: 8b-instruct) to extract information, and assembles an applicant dossier. Outputs will be checked against the sources, with unclear or conflicting information flagged. Personal essays and similar subjective documents remain outside AI evaluation.

An admissions officer reviews the dossier, resolves flagged issues, reads essays, and makes the final decision. Missing evidence or failed output validation should stop automated continuation and require human attention.



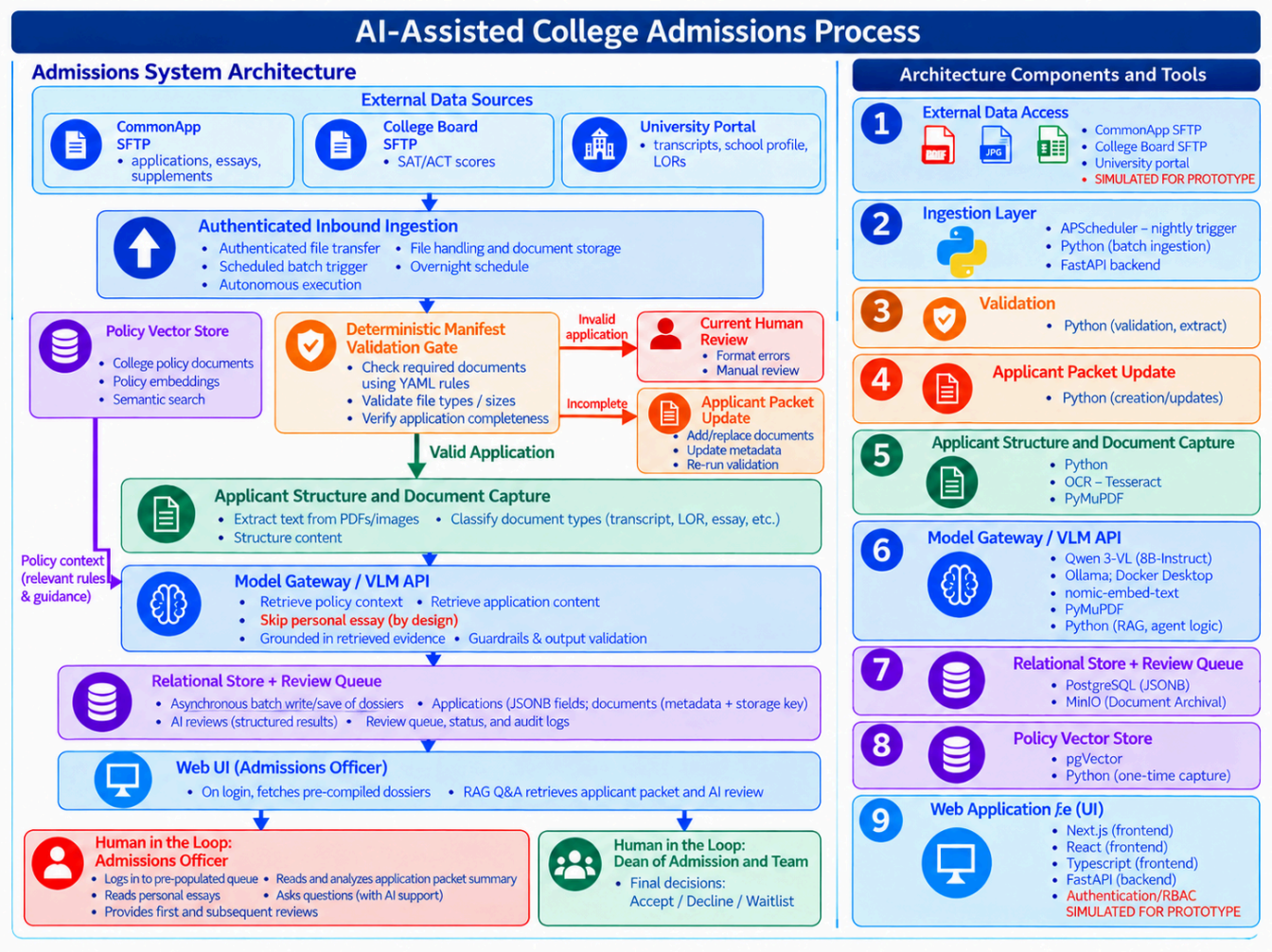
III. Step Contracts & State Handling

Step	Input	Operation / Responsibility	Output	Validation & Failure Behavior
Ingestion: 1. Ingestion Pass 1: Applicant Record Staging	Application CSV	Select applicants belonging to the batch, normalize fields, and create or update applicant records.	Structured applicant records with PENDING status	Rows without applicant IDs are skipped. Completeness is checked later. Database failures can fall back to in-memory storage.
2. Ingestion Pass 2: Document Linking and Archival	Staged applicant IDs and directory of PDFs and documents	Match files to applicants, classify document types, check basic integrity, calculate checksums, and archive files.	Linked document metadata, storage keys, integrity findings, and orphan records.	Check PDF extension, size, and header. Retain detected issues and unmatched files. Archival may fail or be simulated; a returned key doesn't confirm storage.
3. Manifest Validation Gate	Applicant metadata and document inventory	Check required fields and document types, and check review count against the configured .yaml file.	Routing status (ready, waiting, incomplete, or error) plus missing-item/error details.	Integrity errors take priority. Only packets passing the gate proceed to summarization. Intake does not establish document legibility.

4. Intake Reporting and Handoff	Batch validation results	Update applicant statuses, generate ingestion reports, and export eligible IDs.	Ingestion report and affected_ids.json.	Export failures stop execution. Successful intake deliberately stops before the AI agent.
Summarizing Agent: 5. Applicant Selection	Applicant ID; PostgreSQL records	Select applicants with READY_FOR_REVIEW status for summarization	Applicant processing context	Skip ineligible or missing records; --force permits regeneration
6. Document Retrieval	Applicant ID; MinIO PDFs	Retrieve permitted PDFs and render pages using PyMuPDF	Document pages for Qwen3-VL	Flag missing or inaccessible documents; exclude personal essays
7. Policy Retrieval (RAG)	Applicant data; pgvector policy store	Retrieve relevant admissions policies using vector embeddings	Policy context and references	Flag missing or insufficient policy evidence
8. VLM Summarization	Applicant records, PDFs, policy context	Use Qwen3-VL through Ollama to generate six structured dossier sections	AI-generated summaries and citations	Retry failed sections once; flag persistent failures
9. Evidence Validation	Generated sections and source evidence	Validate JSON structure, citations, restricted content, and policy-evidence alignment	Validated dossier sections	Reject unsupported citations or invalid sections
10. Dossier Persistence	Validated dossier	Save dossier, generation history, and audit logs to PostgreSQL	DOSSIER_READY; persisted dossier	Record failures; do not mark invalid dossiers as ready
Chat UI: 11. Applicant Selection	Applicant ID; officer selection	Retrieve applicant information through Next.js API routes	Selected applicant record	Display error or empty state for unavailable applicants
12. Dossier Retrieval	Applicant ID; PostgreSQL	Retrieve AI-generated dossier and supporting evidence	Dossier displayed in review panel	Handle missing dossiers and database errors
13. Document Viewing	Applicant ID; MinIO document references	Retrieve and display original applicant PDFs	PDF document viewer	Display retrieval errors when documents are unavailable
14. Chatbot Q&A	Officer question; applicant dossier + docs.	Use Ollama and Qwen3-VL for applicant-specific, evidence-grounded Q&A	AI-generated response with supporting evidence	Avoid unsupported answers and conclusions
15. **Human Review**	Dossier, PDFs, chatbot.	Provide independently scrollable review panels	Interactive Admissions Officer workspace, human assessment	Handle loading and error states, while preserving applicant context

The system maintains state across the workflow using applicant IDs, processing statuses, document metadata, validation results, generated dossier sections, error messages, and retry counts. PostgreSQL stores structured applicant records, workflow status, validated AI outputs, and audit information, while MinIO stores the original supporting documents. During processing, each step updates the applicant's status and records intermediate results so downstream steps know whether the application is eligible to continue. Errors, validation failures, retry attempts, and key processing events are logged so the team can reconstruct what occurred during a run. If processing stops partway through, completed results remain persisted and the application can be retried or resumed from the appropriate step; cases that cannot be safely continued are routed for manual review rather than restarting the entire workflow.

IV. Updated Architecture & Prototype



We successfully tested a prototype starting with an automated nightly ingestion using **APScheduler** that for now demonstrates intake through a manual CLI trigger. The first pass reads the batch's applicant flat file (CSV) and identifies each applicant using their application ID, organizes the structured information, including demographics, academic fields, activities, awards and AP coursework and creates applicant records in **PostgreSQL** and assigns an initial status of "PENDING". The second pass scans applicant folders in the batch directory, matches each document to a staged applicant using the application ID in the filename and with the existing **PostgreSQL** applicant database, then classifies the document by its filename. The storage manager handles archival of the original files in **MinIO** and attaches document **JSONB** metadata to the applicant record. Unmatched files are retained separately as "orphan" documents while detected file issues are recorded for subsequent validation. After both passes, the manifest validation gate evaluates required applicant information completeness, document completeness, and recorded file errors, assigns a routing status and passes an "affected-ids" list for the AI summarizing agent to process only applications that need update from the latest batch. **Trust Boundary 1** treats incoming CSVs and PDFs as untrusted. **Trust Boundary 2** is the intended perimeter around internal storage and processing; access controls, network isolation, and immutable logging are design requirements rather than verified controls in the local demonstration.

The summarizing agent, a Python AI processing component that uses **Qwen3-VL (8B-Instruct)** and runs locally through **Ollama**, then generates structured, evidence-grounded applicant dossiers. The agent retrieves applicant metadata from **PostgreSQL** and supporting PDF documents from **MinIO object storage**. It uses **PyMuPDF** to prepare document content, including image-based transcripts, for

vision-language processing. The architecture implements **Retrieval-Augmented Generation (RAG)** using **PostgreSQL** with the **pgVector** extension. Institutional admissions policies are embedded using **nomic-embed-text** and retrieved to provide relevant policy context during evaluation. The agent generates six structured sections covering academics, engagement, recommendations, supplemental materials, policy alignment, and overall synthesis. Reliability controls include section-level retries/error checking, evidence citation tracking, restricted-content filtering, policy-to-evidence validation, and exclusion of personal essays from AI evaluation. Validated dossiers, generation records and audit information are persisted in **PostgreSQL**. The agent operates as a backend processing component rather than making autonomous admissions decisions.

An Admissions Review Chat UI using web-based **Next.js, React,** and **TypeScript** frontend then provides Admissions Officers with an integrated interface for reviewing applicant information, AI-generated dossiers, raw document views, and an interactive question-and-answer assistant.The architecture uses **Next.js server-side API routes** to communicate with backend data sources, including **PostgreSQL** for applicant records and generated dossiers and **MinIO** for supporting PDF documents. The interface is organized into three independently scrollable panels, allowing officers to navigate application documents, review AI-generated findings, and interact with the chatbot without losing their place in the applicant record. The chatbot is designed to support applicant-specific, evidence-grounded questions using available dossier information and supporting documentation. This interface serves as the human-in-the-loop review layer, enabling Admissions Officers to inspect AI-generated assessments and their supporting evidence while retaining responsibility for human judgment and review.

V. Initial Evaluation & Design Decisions

The initial Batch01 run had 10 applicants totaling 100 PDFs. The case results are as follows:

Test Input / Case	Expected Behavior	Actual Behavior	Evidence / Metric	Interpretation & Next Action
Full Application (APP_001)	Ingestion with no errors and AI agent produce completed dossier.	READY_FOR_REVIEW; 10 PDFs linked; ID exported. This intake run stopped before the AI agent.	Batch report lists 10 documents, no issues, and APP_001 in the ready-ID file.	Intake and handoff worked. Evaluate dossier generation as a separate test.
Missing Documents (APP_010)	Route to application needs update; ID excluded from handoff to AI agent.	AWAITING_MATERIALS; transcript listed as missing; ID excluded from the ready-ID file.	Nine PDFs recorded; APP_010 absent from affected_ids.	Missing documents branch works in stopping application from proceeding. No emails generated yet.
Obscured transcript content (APP_008)	Stops processing continuation and route to human review.	READY_FOR_REVIEW at intake. The PDF passed file-header and size checks, although its contents are intentionally obscured.	Ten PDFs recorded; APP_008 appears in the ready-ID file. The fixture contains a document-quality alert.	Ingestion checks file integrity and presence, not whether grades are legible. Add a downstream readability check and flag the transcript for human attention.
Invalid PDF header (controlled test)	Route a file-integrity failure to Human Review and stop its handoff.	The gate returned ERROR and routed the constructed invalid document to Human Review.	Unit test checks an invalid PDF header and MIME type. The regular 10-applicant batch had zero errors.	The error route is covered by a controlled test. The error routing branch is working.

The second Batch02 run had 10 applicants with multiple test cases and redundant document pdfs in packet folders removed. The test summary is as follows:

Test Input / Case	Expected Behavior	Actual Behavior	Evidence / Metric	Interpretation & Next Action
APP_011 — Invalid GPA, missing transcript, and null IB count. GPA values 96/102; transcript absent; IB count missing.	Detect or normalize GPA, identify incomplete document packet, and handle absent IB coursework.	Insert failed because GPA exceeded numeric precision. Later insert failed because total_ibs was null.	NumericValueOutOfRange; NotNullViolation on total_ibs; 4 source PDFs.	Correct GPA and IB defaults; verify incomplete-document classification.
APP_012 — Complete packet and downstream summarization. Five PDFs; image-based transcript; CommonApp includes essay.	Ingest metadata, archive documents, establish readiness, and support evidence-grounded summarization without personal essay.	Downstream Qwen3-VL summarizer successfully processed APP_012 and saved dossier.	6 sections passed; 121 citation checks; 0 review notes; DOSSIER_READY.	Confirmed downstream summarization. Document ingestion separately.
APP_013 — Standard five-document packet: CommonApp, transcript, two recommendations, supplement.	Match CSV ID to folder, validate PDFs, archive to MinIO, and assign status.	Five source PDFs present; individual final ingestion status not confirmed.	5 files listed under batch_02/APP_013/.	Verify PostgreSQL status, document count, and MinIO keys.
APP_014 — Standard five-document packet with applicant-specific content.	Preserve metadata, associate all documents, and complete manifest check.	Five source PDFs present; no confirmed final status or individual error.	5 files listed under batch_02/APP_014/.	Check no cross-applicant document association.
APP_015 — Five-document packet with image-based transcript.	Validate/store PDF even if text is not extractable at ingestion.	Five source PDFs present; final ingestion outcome not confirmed.	CommonApp, transcript, two letters, supplement.	Verify PDF validation and MinIO storage; VLM reading is downstream.
APP_016 — Null mailing address and five-document packet.	Handle missing metadata per completeness rules w/o unhandled DB error.	PostgreSQL rejected insert because mailing_address was null and column was NOT NULL.	NotNullViolation on mailing_address.	Align database nullability and incomplete-applicant routing.
APP_017 — Standard five-document packet.	Process independently, verify PDFs, upload to MinIO, and classify completeness.	Five source PDFs listed; final ingestion status not verified.	5 files listed under batch_02/APP_017/.	Check final status and document persistence.
APP_018 — Five-document packet with stored metadata inspection.	Preserve document identity, checksum, and storage metadata.	Source files and document metadata observed; final application status not established.	MinIO object keys and SHA-256 metadata present.	Verify completeness classification and accessibility of all five objects.
APP_019 — Standard five-document packet.	Validate presence/format, archive files, and update review status.	Five source PDFs present; final individual status not confirmed.	5 files listed under batch_02/APP_019/.	Check report and PostgreSQL record.
APP_020 — Final applicant in Batch 02; five-document packet.	Complete last applicant without skipping files or mixing records.	Five source PDFs present; successful final ingestion not independently verified.	5 files in batch_02/APP_020/	Confirm last record, correct document count, and status.

DDR 1: Deterministic Validation Before AI Processing (M1)	
Decision	Retain rule-based validation to check completeness and requirements before AI analysis. (M1 DDR 1)
Rationale	Objective checks have clear answers and can be applied consistently through ordinary code. Catching missing or unsuitable files early can prevent unnecessary AI processing.
Alternative	Use an AI model to decide whether an application is complete and meets the basic requirements.
Trade-off	Rule-based checks are predictable and easy to test, but the rules themselves need updating and maintaining when requirements change. Also, rule-based checks cannot resolve every exception or determine whether the file's contents are readable and accurate.
Evidence	The prototype's implemented gate was able to block APP_010 because its transcript was missing. In five controlled ingestion cases, the current gate produced all five expected routes, while a simpler filename checklist produced four. However, APP_008 passed intake despite its unreadable transcript, showing that further checks are needed downstream. In Batch02 results, this was all resolved (APP_011).
DDR 2: VLM over LLM	
Decision	Utilizing a Vision-Language Model over a Large Language Model to process visually structured documents; excluding personal statements and essays.
Rationale	Applicant documents contain tables, formatting, signatures, scanned text, and other visual elements that may be lost when documents are converted into plain text. A VLM can evaluate both the text and visual layout of each page, which may improve the extraction of courses, grades, and other context needed to create the applicant dossier.
Alternative	Extract text using PyMuPDF/OCR and send it to a text-only LLM. This approach would be simpler and require fewer image-processing steps, but it may lose table structure, page layout, and information contained in scanned or visually complex documents
Trade-off	The VLM can preserve more visual and layout information. Still, it requires converting PDF pages into images and generally uses more processing time and model resources than a text-only LLM. Image conversion or poor scan quality may also introduce errors. The system must validate the VLM's JSON output and route any unreadable or incomplete results to human review.
Evidence	On APP_001, Gemma 3 VLM processes 10 rendered pages from eight permitted PDF docs. In 61.5 seconds, 3/4 dossier sections passed the validation; the engagement section failed exact-citation validation after two attempts and was correctly routed to human review. In comparison, Qwen3.5 completed the same case in 56.2 seconds but passed only 2/4 sections. However, a controlled comparison with a text-only LLM is still needed to establish whether the VLM is more accurate than the LLM only alternative via PyMuPDF text capture. Further tests to be done to compare to an LLM.
DDR 3: Hybrid Relational JSONB Schema with Selective Ingestion Field Allowlisting	
Decision	Store core applicant demographics in 33 flat relational PostgreSQL columns while storing document metadata, AP courses, awards, and activities in JSONB arrays, enforced by selective column allowlist (CSV_UPDATE_FIELDS) during multi-batch upserts
Rationale	Fixed relational columns enable quick SQL filtering and strict schema constraints (GPA ranges, unique applicant IDs), while JSONB allows variable-length document metadata and external score feeds to attach without DDL migrations. Restricting CSV updates though permit-list of approved fields ensures that subsequent CommonApp demographic updates cannot overwrite or erase previously attached document arrays or external test scores
Alternative	A monolithic single-table JSONB document store (storing the entire application dossier as one nested JSON object) or an entirely normalized relational schema (using separate join tables for document records, activities, and awards)
Trade-off	Relational columns require rigorous data-cleaning, nullability checks, and type-conversion handling during Pass 1 ingestion(Batch 02 numeric and null constraint errors). This makes downstream context assembly really simplified, requiring SELECT query without multi-table joins or fragile JSON unnesting

Evidence	In Batch 2 testing, unconstrained flat-file parsing exposed the necessity of schema bounds when AP_011 triggered NumericValueOutOfRange on raw GPA values (96/102) and APP_016 triggered NotNullViolation on mailing_address. Implementing selective update filters in CSV_UPDATE_FIELDS and packets during Pass 1 allowed 61 automated integration tests to pass (pytest tests/ -v), successfully proving that repeated CSV drops for APP_042 updated demographic fields while preserving all 5 attached PDF records and READY_FOR_REVIEW status

Design decisions “Batch Processing Cadence” from Milestone 1 remains untested for now. The focus of M2 was to get the system working on a small batch of sample data so testing on full data has not yet been executed and will remain a task for later milestones.

VI. Contribution & Next Steps

Member	Completed Contribution	Evidence Location	Result / Impact
Danny McIntosh	Built the intake, storage, validation, testing, and handoff pipeline; supported architecture documentation.	Commits aa7541a, 4232a05; intake/tests/README, M2 report.	Created the deterministic intake and routing foundation for downstream AI processing.
Maria Malhotra	Designed HLD/LLD architecture/database, implemented policy RAG, co-implemented agent and UI development, ingestion fixes, developed test cases, led full integration testing.	Commits 8cf60df, 847ed1b, bb562d7, a0ba0c5; AdmitGPT seed app file, pgVector, Batch 02 test cases, archi docs, M2 report.	Enabled successful end-to-end system integration, data ingestion reliability, UI design, validation of overall prototype functionality and design
Maya Patel	Created synthetic applicants/documents, initial M2 UI, developed M3 UI, performed integration and LLM comparison testing	Commit 71c4e98, 000ef79; test data, initial UI, work for future triage-screen UI	Expanded test coverage and initial development of the human-review interface.
Mi Le	Designed the orchestration workflow, implemented packet-folder ingestion, and performed workflow/performance testing.	Commits 5f59c54, 4169f66, 48077f4; workflow diagram, ingestion code, test results, M2 report.	Defined system flow and helped verify ingestion and orchestration behavior.
Tran Le	Co-developed summarizing agent, benchmarked Qwen3-VL models, co- built the Officer Review Portal/UI chatbot, performed full pipeline testing.	Commits 0f3d5e6, f71fb4f, c9ac065, 01f6b83, 1cc8c31; benchmark and UI artifacts.	Supported VLM selection and connected AI-generated dossiers to the officer review experience.

Building on the Milestone 2 prototype, the team has identified several potential next steps contingent upon available time. Near-term priorities include expanding Pass1/Pass2 ingestion testing across more combination cases; testing College Board SAT/AP and ACT scores ingestion; handling exception cases; validating audit-log behavior; and some expanded UI capabilities such as dashboard views, final decision controls (Accept, Waitlist, or Decline), and an exception-triage screen. Planned enhancements include handling IB coursework, high school profiles and orphan documents, automated notifications for applicants with missing materials, and implementing simulated authentication and role-based access control. The team plans to test the fully integrated pipeline through unattended APScheduler batch execution and the complete Next.js review interface.

Future work beyond the course would focus on scalability and broader admissions use cases. This includes benchmarking batch-processing cadence, memory usage, and extraction throughput at larger institutional volumes; retaining historical admissions data for cross-cycle comparisons; and extending the system to support transfer and international student applications. AI tools assisted with development of code scripts and implementation of technical details. The team reviewed and ran tests and evaluation runs manually. Observed system behavior was checked against repository code, PDFs, saved results and logs, and team clarifications.